

Let's create a .NET Core console app that performs data access against a SQL Server database using EF Core. We can try the same using Visual Studio on Windows, or by using the .NET Core CLI on Windows, macOS, or Linux with VSCode as a free editor. I have copied the complete sample at <https://github.com/Maheshk-MSFT/efcorecodefirstapproach> as a reference.

In this sample, C# models created in our local project can be used as a schema to generate database tables. Assuming you have .NET Core installed along with entity framework runtime, execute the steps to get the models translated into databases.

Here's a quick test to confirm if we have .NET Core and EF Core installed.

In this first sample, we have created four model classes – Customer, Order, Product and ProductOrder – along with their relationships.

After setting the required SQL Database Connection string, we need to start initialising the local model classes to actual database tables. Execute the commands given below in a sequential order:

a) `dotnet ef migrations add InitialCreate`

The above command initialises the local model classes prepared for migration.

b) `dotnet ef database update`

This command helps us to generate equivalent tables in the database.

c) `dotnet ef migrations add AddBlogCreatedTimestamp`

The sub-command *Add* helps to amend incremental changes synced with actual DB tables.

d) `dotnet ef database update`

This command helps to update the local changes.

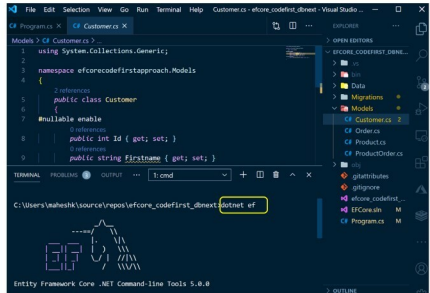


Figure 4: VS Code Editor Dotnet CLI

Data Context: The Interface to Data

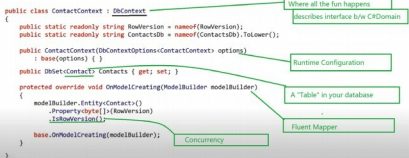


Figure 5: A closer look at Data Context

Now, let's see what the Data Context class is and its role. We need to configure the provider in our *DbContext*, either using the *OnConfiguring* method or the *AddDbContext* method, if we are using a dependency injection container. For example, the following line configures the SQL Server provider with the passed connection string:

```
optionsBuilder.UseSqlServer(
    "Server=(localdb)\\mssqllocaldb;Database=MyDatabase;Trusted_Connection=True;");
```

Failing to set the right *connectionstring* will throw an error

asking to fix the connection first. Figure 5 explains the Data Context class and its various functionalities.

We can initialise the Data Context class in *program.cs* for making the database call. In Figure 6, you can see that we are reusing the same *datacontext* 'context' for creating, updating or deletion of the records. The Data Context class facilitates the connection between our local .NET Class and its memory state to database records.

On executing the final project using `dotnet run program.cs`, one can see the record creation, updation and deletion at a glance. I suggest you check out the complete code for this topic and try it.